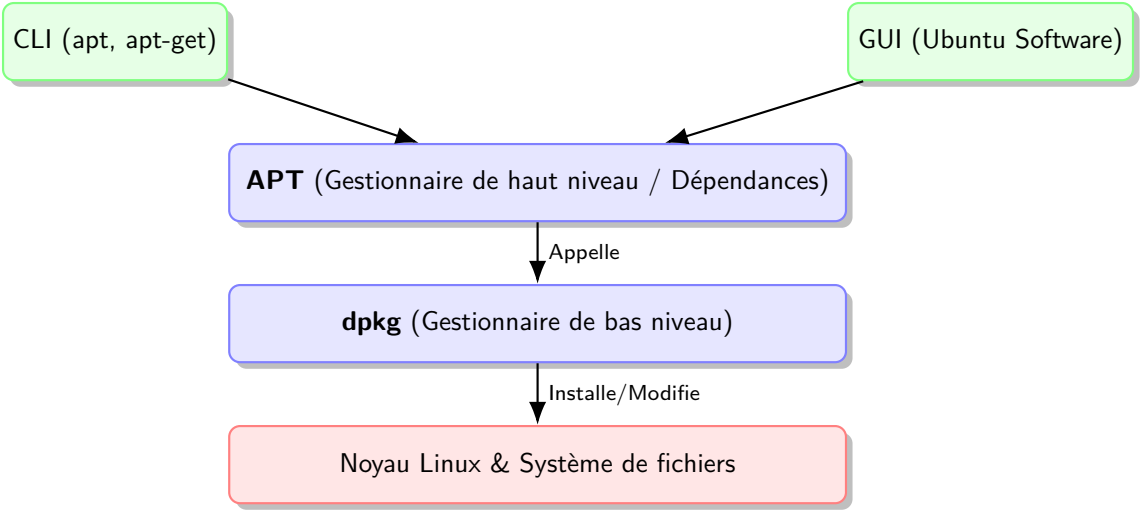
Chapitre 4

Gestion des paquets

Qu'est-ce qu'un paquet ?

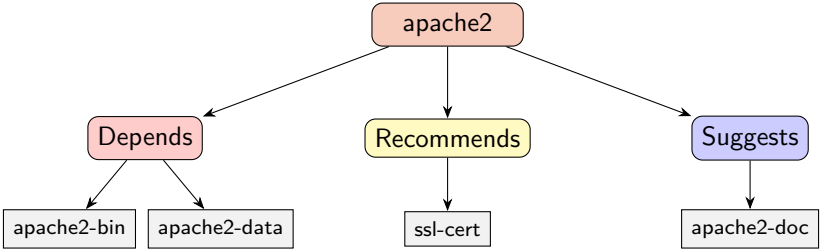
- **Définition :** Un paquet est une archive compressée contenant tous les fichiers nécessaires au fonctionnement d'un logiciel.
- **Contenu typique d'un paquet (`.deb`) :**
  - Les binaires exécutables.
  - Les fichiers de configuration.
  - La documentation (pages `man`).
  - Les métadonnées (version, mainteneur, **dépendances**).
- **Pourquoi ne pas compiler depuis les sources ?**
  - Gain de temps considérable.
  - Stabilité et tests garantis par la distribution.
  - Mises à jour centralisées.

Architecture de gestion des paquets



APT résout les dépendances et télécharge, dpkg installe localement.

L'arbre des dépendances



Depends Requis absolument. Le paquet ne fonctionne pas sans.

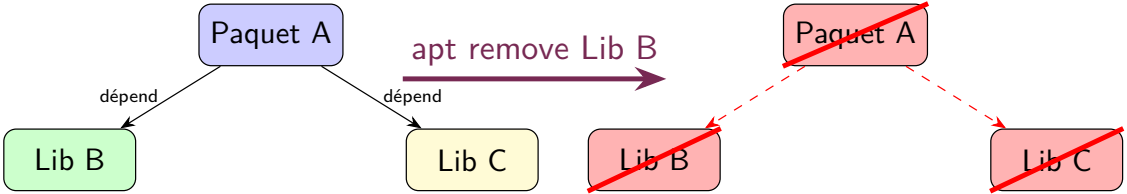
Recommends Fortement suggéré.

Suggests Optionnel. Améliore l'utilité.

Résolution des dépendances

Le danger des suppressions en cascade

Supprimer une dépendance critique peut désinstaller plusieurs paquets essentiels.



État Stable

Suppression en Cascade

dpkg : Le cœur du système Debian/Ubuntu

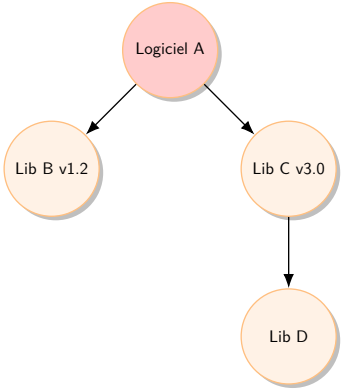
Rôle de dpkg

Outil historique permettant d'installer, construire, supprimer et gérer les paquets `.deb` locaux.

Commandes essentielles :

- `sudo dpkg -i paquet.deb` : Installe un paquet local.
- `sudo dpkg -r nom_paquet` : Supprime un paquet (garde la configuration).
- `sudo dpkg -P nom_paquet` : Purge un paquet (supprime tout).
- `dpkg -l` : Liste tous les paquets installés.

La limite de dpkg : problème des dépendances



- `dpkg` **ne télécharge pas** les dépendances.
- Si vous tentez d'installer "A" sans "B", l'installation échoue.
- C'est ici qu'intervient **APT**.

APT (Advanced Package Tool)

- **Rôle principal :** Résoudre automatiquement les dépendances et interroger les dépôts distants.
- **La différence entre `apt` et `apt-get` :**
  - `apt-get` : L'outil historique, robuste, parfait pour les scripts.
  - `apt` : Plus récent (Ubuntu 16.04+), combine `apt-get` et `apt-cache` avec une interface utilisateur plus agréable (barre de progression, couleurs).

APT : fichiers sources

- /etc/apt/sources.list
- /etc/apt/sources.list.d/
- Format : deb [options] URI distribution composante1 composante2

Exemple Linux Mint : /etc/apt/sources.list.d/official-package-repositories.list

```
deb https://mirror.marwan.ma/ubuntu noble main restricted universe multiverse
deb https://mirror.marwan.ma/ubuntu noble-updates main restricted universe
    multiverse
deb https://mirror.marwan.ma/ubuntu noble-backports main restricted universe
    multiverse

deb http://security.ubuntu.com/ubuntu/ noble-security main restricted universe
    multiverse
```

Dépôts Ubuntu

Les paquets Ubuntu sont classés en 4 composantes principales selon leur niveau de support et leur licence :

- **Main** : Logiciels libres et open-source supportés officiellement par Canonical.
- **Universe** : Logiciels libres maintenus par la communauté.
- **Restricted** : Pilotes propriétaires pour les périphériques (ex : Nvidia).
- **Multiverse** : Logiciels non libres ou soumis à des restrictions légales (codecs).

Les PPA (Personal Package Archives)

Qu'est-ce qu'un PPA ?

Un dépôt non officiel hébergé sur *Launchpad*, permettant aux développeurs de distribuer des versions plus récentes de leurs logiciels que celles des dépôts officiels.

Utilisation

1. `sudo add-apt-repository ppa:nom/ppa`
2. `sudo apt update`
3. `sudo apt install logiciel`

Attention : Ajouter un PPA nécessite d'avoir confiance en son mainteneur (risque de sécurité ou de casse des dépendances).

PPA : Exemple concret

Installation de « **Inkscape** » via PPA

```
# Ajouter le PPA officiel d'Inkscape
sudo add-apt-repository ppa:inkscape.dev/stable
sudo apt update
sudo apt install inkscape
```

Permet d'ajouter le fichier :
`/etc/apt/sources.list.d/inkscape_dev-stable-noble.list`

qui contient la ligne :

```
deb [signed-by=/etc/apt/keyrings/inkscape_dev-stable-noble.gpg] https://ppa.launchpadcontent.net/inkscape.dev/stable/
    ubuntu noble main
```

Ubuntu 24.04 : évolution de la configuration APT

- Ubuntu 24.04 introduit des changements majeurs dans la gestion des sources et des clés.
- **Objectifs :** sécurité, maintenabilité, standardisation (Debian).
- **Principales évolutions :**
  - Adoption du format **deb822** pour les sources.
  - Fichiers `.sources` dans `/etc/apt/sources.list.d/`.
  - Abandon de `apt-key` au profit de `signed-by`.
  - Utilisation de `/usr/share/keyrings/` pour stocker les clés.

Le format deb822

Ancien format (une ligne)

```
deb http://archive.ubuntu.com/ubuntu noble main restricted
```

Nouveau format deb822 (multi-lignes)
fichier : /etc/apt/sources.list.d/ubuntu.sources

```
Types: deb
URIs: http://ma.archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

- Plus lisible et extensible.
- Permet d'ajouter des options comme `Signed-By`, `Architectures`, etc.

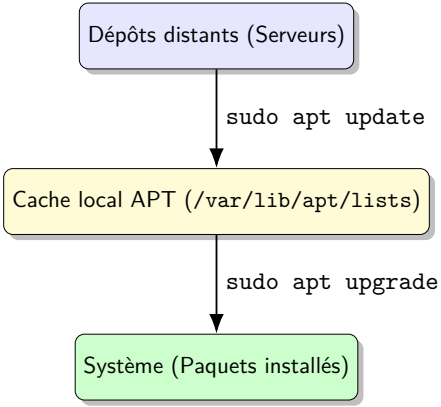
Fichiers .sources dans sources.list.d

- Le fichier principal /etc/apt/sources.list reste au format une ligne (par compatibilité).
- Les dépôts tiers doivent être placés dans /etc/apt/sources.list.d/ avec l'extension .sources et au format deb822.

Exemple : dépôt Docker
Fichier /etc/apt/sources.list.d/docker.sources :

```
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Signed-By: /usr/share/keyrings/docker.gpg
```

Cycle d'une mise à jour APT



1. **Update :** Met à jour l'index local depuis les serveurs.
2. **Upgrade :** Compare l'index avec le système et installe les nouvelles versions.

upgrade vs full-upgrade

- **apt upgrade** : installe les mises à jour sans retirer de paquets ni modifier les dépendances de façon majeure.
- **apt full-upgrade** (ou dist-upgrade) : peut supprimer des paquets pour résoudre des conflits et effectuer des changements non triviaux.

Commandes usuelles APT

Commande	Action
`apt search [mot]`	Recherche un paquet dans le cache.
`apt show [paquet]`	Affiche les détails (version, dépendances).
`sudo apt install [p]`	Installe un paquet et ses dépendances.
`sudo apt remove [p]`	Supprime un paquet.
`sudo apt autoremove`	Nettoie les dépendances orphelines.

Les options de filtrage et recherche

- **`apt list --installed`** : Liste uniquement les paquets présents sur le système.
- **`apt list --upgradable`** : Affiche les paquets qui attendent une mise à jour (très utile avant un upgrade).
- **`apt-cache depends [paquet]`** : Visualise l'arbre des dépendances d'un logiciel avant installation.
- **`apt-cache rdepends [paquet]`** : *Reverse depends* — voir quels autres paquets ont besoin de celui-ci (crucial avant une suppression).

Astuce de recherche

`apt search [mot-clé] | grep "^paquet"` : Cherche un paquet et filtre pour ne voir que ceux qui ne sont pas encore installés.

Gestion des versions : Le "Pinning"

Parfois, on souhaite installer une version spécifique ou empêcher une mise à jour.

- **Installer une version précise :**
  - `sudo apt install [paquet]=[version]`
  - Exemple : `sudo apt install docker-ce=5:24.0.7-1`
- **Bloquer une mise à jour (Hold) :**
  - `sudo apt-mark hold [paquet]` : Empêche le paquet d'être mis à jour lors d'un *upgrade* général.
  - `sudo apt-mark unhold [paquet]` : Relance le suivi des versions.

Nettoyage et Maintenance du Cache

Le système accumule des fichiers temporaires dans `/var/cache/apt/archives/`.

apt clean

Supprime **tous** les fichiers .deb téléchargés du cache. Libère beaucoup d'espace.

apt autoclean

Supprime uniquement les paquets obsolètes (qui ne sont plus dans les dépôts).

apt autoremove

Supprime les dépendances qui ont été installées automatiquement mais ne sont plus nécessaires.

Options de sécurité et de forçage

À utiliser avec prudence

Ces options permettent de passer outre les sécurités standards d'APT.

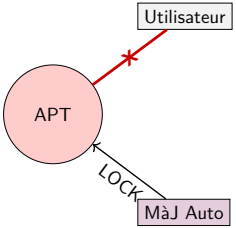
- `--fix-broken install` (`-f`) : Tente de réparer un système où les dépendances sont corrompues.
- `--dry-run` (`-s`) : Simule l'action sans rien modifier au système (indispensable pour les serveurs critiques).
- `-y` : Répond automatiquement "Oui" à toutes les questions (pratique pour l'automatisation).
- `--no-install-recommends` : Installe uniquement les dépendances strictes, sans les paquets "suggérés" ou "recommandés" (gain de légèreté).

Erreur : "Could not get lock"

Le problème : Un seul processus peut utiliser APT à la fois pour éviter la corruption de la base de données.

Causes courantes

- Mise à jour automatique en arrière-plan.
- Logiciel "Ubuntu Software" ouvert.
- Terminal fermé brutalement pendant un « `apt install` ».



Solution recommandée

Attendre 1 à 2 minutes ou vérifier le processus fautif :

```
ps aux | grep -i apt
```

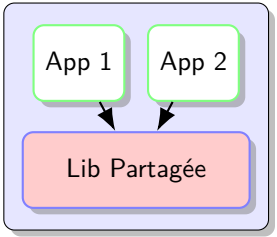
L'évolution : Pourquoi Snap ?

- Les paquets classiques (.deb) partagent leurs dépendances.
- Si le logiciel A a besoin de la LibC v1, et le logiciel B de la LibC v2, des conflits surgissent.
- **La solution Canonical : Snap.**

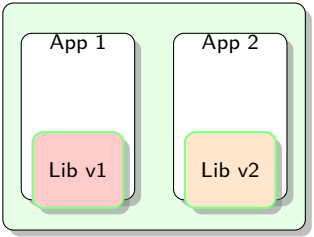
- **Avantages de Snap :**
  - **Conteneurisation** : Le paquet contient toutes ses dépendances.
  - **Isolation** : L'application tourne dans un bac à sable (Sandboxing).
  - **Mises à jour automatiques** et invisibles.
  - **Universel** : Fonctionne sur presque toutes les distributions Linux.

Comparaison : APT vs Snap

Modèle APT (.deb)



Modèle Snap



- Les Snaps prennent plus d'espace disque.
- Mais ils évitent totalement le "problème des dépendances".

Commandes de base Snap

- `snap find [logiciel]` : Chercher un snap.
- `sudo snap install [logiciel]` : Installer une application.
- `snap list` : Voir les snaps installés.
- `sudo snap refresh` : Forcer la mise à jour des snaps.
- `sudo snap remove [logiciel]` : Désinstaller.

Note

Sur les versions récentes d'Ubuntu, le logiciel "Ubuntu Software" (App Center) installe des paquets Snap par défaut.

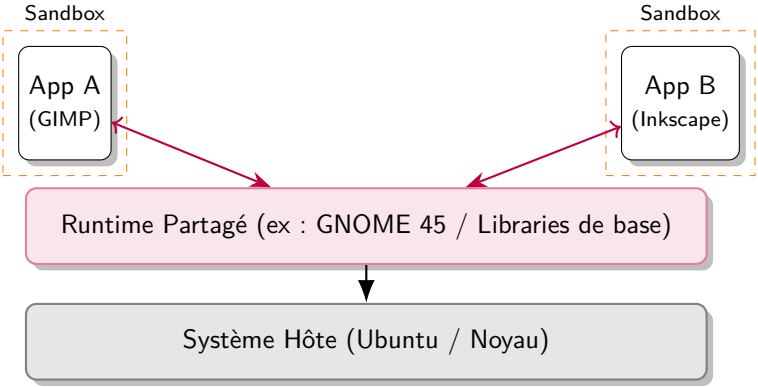
Qu'est-ce que Flatpak ?

- **Origine :** Développé par la communauté (Freedesktop.org), très populaire sur Fedora, Mint et Debian.
- **Philosophie :** Focus sur les applications "Desktop" (graphiques) plutôt que les services serveurs.
- **Concept de Runtime :** Contrairement à Snap qui embarque tout, Flatpak partage des "couches de base" (ex : Runtime GNOME ou KDE) entre plusieurs applications pour gagner de l'espace.

Le magasin principal : Flathub

C'est le dépôt central (quasi universel) où l'on trouve la majorité des applications Flatpak.

Architecture Flatpak : Le Runtime partagé



Le Runtime réduit la redondance : plusieurs apps utilisent la même base.

Utilisation de Flatpak sur Ubuntu

- **Installation :** Par défaut, Ubuntu ne propose pas Flatpak.
  - `sudo apt install flatpak`
  - `flatpak remote-add --if-not-exists flathub https://flathub.org/repo/flathub.flatpakrepo`
- **Commandes clés :**
  - `flatpak install flathub [ID_APP]` : Installation.
  - `flatpak run [ID_APP]` : Lancement.
  - `flatpak update` : Mise à jour globale.

Snap vs Flatpak

Critère	Snap (Canonical)	Flatpak (Libre)
Backend	Fermé (propriétaire)	Ouvert (libre)
Vitesse	Démarrage parfois lent	Plus rapide (Runtimes)
Permissions	Gérées par Canonical	Gérables par **Flatseal**
Usage	Desktop + Serveur + IoT	Desktop uniquement
Updates	Automatiques/Forcées	Au choix de l'utilisateur

Bonnes pratiques de gestion

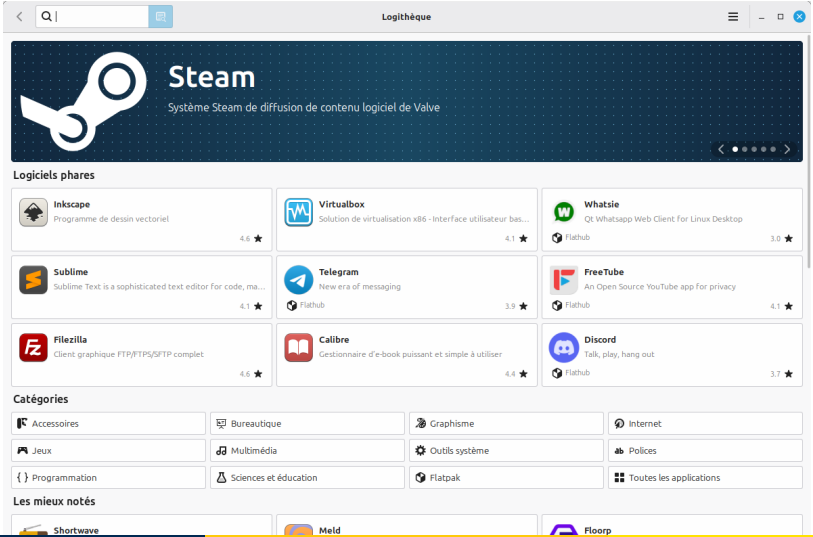
1. Toujours faire un `apt update` avant d'installer un nouveau logiciel.
2. Nettoyer régulièrement le système avec `sudo apt autoremove`.
3. Limiter le nombre de PPA tiers pour garantir la stabilité du système.
4. Privilégier APT pour les utilitaires systèmes et Snap/Flatpak pour les applications de bureau complexes (navigateurs, IDE).
5. Ne jamais forcer l'installation de dépendances cassées sans comprendre le message d'erreur d'APT.

Résumé

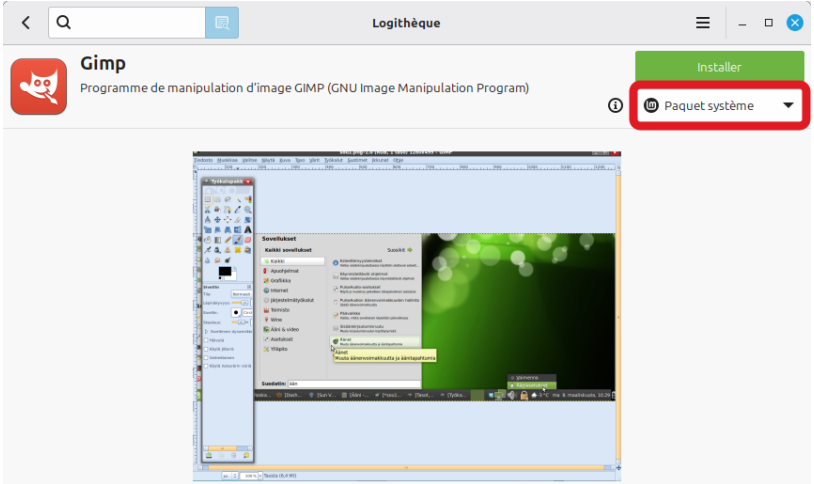
Outil	Niveau	Cas d'usage principal
dpkg	Bas	Fichiers `.deb` locaux hors-ligne
APT	Haut	Gestion fluide, dépôts, dépendances
Snap	Isolé	Applications embarquant leurs dépendances
Flatpak	Isolé	Apps desktop (Flathub, respect du libre)

- **Système :** Utilisez **APT** pour le noyau et les outils de base.
- **Applications : Snap** ou **Flatpak** pour les versions récentes (Discord, LibreOffice, IDE).
- **Urgence : dpkg** pour un paquet récupéré manuellement.

Installation en mode graphique



Installation en mode graphique
Paquet système



Installation en mode graphique
Paquet Flatpack

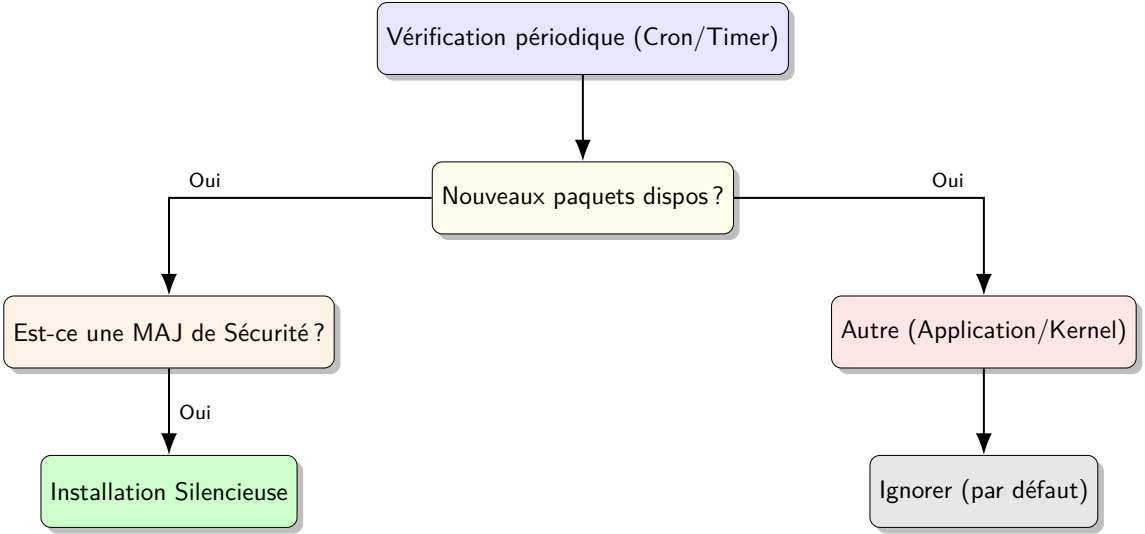
Pourquoi automatiser les mises à jour ?

- **Sécurité :** Réduire la fenêtre d'exposition aux vulnérabilités (0-day).
- **Maintenance :** Éviter l'accumulation de centaines de paquets en retard.
- **Sérénité :** Le système se répare et se protège de lui-même en arrière-plan.

Le service standard : `unattended-upgrades`

Installé par défaut sur Ubuntu Server et Desktop, il gère l'installation automatique des mises à jour de **sécurité** sans intervention humaine.

Logique de décision du service



Configuration et Personnalisation

Les fichiers de configuration se trouvent dans : `/etc/apt/apt.conf.d/`

- **`50unattended-upgrades` :**
  - Choisir quels dépôts autoriser (ex : autoriser *Universe*).
  - `Unattended-Upgrade::Remove-Unused-Dependencies "true";`
  - `Unattended-Upgrade::Automatic-Reboot "false";`
- **`20auto-upgrades` :**
  - Définit la fréquence (Update-Package-Lists "1" = chaque jour).

Commande de test (Simulation) :

Dry-run

```
sudo unattended-upgrade --dry-run --debug
```

Surveillance et Logs

Même si c'est automatique, il faut pouvoir vérifier ce qu'il s'est passé en cas de problème.

Fichiers Logs : `/var/log/unattended-upgrades/`

Historique APT : `/var/log/apt/history.log`

Le cas du redémarrage

Certaines mises à jour (Noyau, libc) nécessitent un redémarrage. L'outil peut être configuré pour redémarrer automatiquement à 04h00 du matin si nécessaire.